COMBINED ENGINEERING PROJECT REPORT

AI-Powered Water Bottle Quality Inspection System

Combined internal report for engineering review, leadership evaluation, and future development handoff

Field	Details
Organization	SEEWISE.ai
Dataset	1,200 water bottle inspection images annotated with LabelImg
Model	YOLOv8 Small (YOLOv8s), 201 epochs, batch size 32, workers 2
Prepared for	Engineering Managers, Technical Leads, CTO / Leadership Team, Product Team, Future Developers
Prepared by	Internship Project Contributor
Report date	May 30, 2026

This report combines the prior project report with the updated dataset preparation, LabelImg annotation, dataset-balancing, YOLOv8s training, and real-time optimization methodology.

Table of Contents

1. Executive Summary

Project overview

The AI-Powered Water Bottle Quality Inspection System is a full-stack industrial computer vision platform developed for SEEWISE.ai. It combines a React web application, Python FastAPI backend, OpenCV frame handling, YOLOv8 Small model inference, CUDA/FP16 acceleration, analytics, history, and reporting outputs to automate water bottle quality inspection.

This rebuilt report combines the previous management-oriented engineering report with the updated dataset and model-training methodology. It is intended for engineering managers, technical leads, CTO/leadership, product stakeholders, and future developers evaluating internship project impact and production readiness.

Key achievements

- Built an end-to-end inspection workflow for webcam, RTSP camera, mobile IP camera, and uploaded video sources.
- Prepared a custom 1,200-image water bottle inspection dataset annotated using LabelImg in YOLO format.
- Resolved dataset imbalance and label-quality issues through validation and balancing work.
- Trained YOLOv8s for 201 epochs with batch size 32 and 2 workers in a GPU-accelerated environment.
- Implemented real-time inspection with GPU-based inference, FPS status, MJPEG output, analytics, CSV export, screenshot capture, and fullscreen review.
- Delivered a system that is useful to both operators and leadership through live monitoring, historical traceability, and business-readable reporting.

Individual contribution: The work demonstrates technical ownership across dataset preparation, model training, full-stack development, real-time stream debugging, performance optimization, analytics, reporting, and user-facing product polish.

2. Problem Statement

Manual water bottle quality inspection is repetitive, difficult to scale, and prone to variation in operator attention, fatigue, lighting conditions, and line speed. The inspection task requires reliable recognition of bottle presence, fill-level state, and label quality while production continues to move.

- Manual inspection creates throughput bottlenecks as production speed increases.
- Inspection events are hard to review later without structured records and exports.
- Production teams need real-time visibility into defects instead of delayed manual summaries.
- Camera-source diversity creates practical engineering challenges for webcam, RTSP, mobile camera, and uploaded video workflows.

Manual limitation	AI-enabled response
Fatigue and subjectivity	Consistent model-driven detection and pass/fail logic
Limited traceability	Inspection history, analytics, screenshots, and CSV reports
Delayed defect discovery	Live camera monitoring with real-time KPIs
Source complexity	Unified camera and upload workflows through one application

3. Dataset Preparation and Annotation Workflow

Dataset collection

The model was developed using a custom dataset of 1,200 annotated water bottle images collected from the production-style inspection setup. Because the operating environment is relatively controlled, with consistent background and camera placement, the dataset effectively represents the intended real-world inspection conditions.

Annotation workflow using LabelImg

- Collected water bottle inspection images from the target setup.
- Manually annotated objects and quality states using LabelImg.
- Generated YOLO-compatible TXT annotation files for training.
- Validated image-to-label correspondence before training.
- Organized training, validation, and testing data for Ultralytics YOLOv8.

Dataset item	Detail
Total images	1,200 water bottle inspection images
Annotation tool	LabelImg
Annotation format	YOLO TXT format
Environment	Controlled industrial inspection setup
Primary classes	bottle, proper_fill, under_fill, over_fill, label_ok, label_torn, label_missing

4. Dataset Imbalance and Data Quality Solution

During the initial training phase, dataset imbalance and annotation inconsistency were identified. Uneven class distribution, missing label files, incomplete labels, and mismatches between images and TXT files risked reducing detection accuracy and increasing false positives or false negatives.

Potential impact

- Bias toward dominant classes.
- Reduced generalization across quality conditions.
- Lower detection accuracy for minority defect classes.
- Training instability caused by invalid or incomplete samples.

Solution implemented

- Developed a custom validation and balancing script.
- Verified image-to-label correspondence.
- Detected missing annotation files and invalid samples.
- Removed invalid training samples.
- Balanced class distribution and standardized annotation quality.

Dataset lesson: The project reinforced that dataset quality, annotation consistency, and class balance often matter more than selecting a larger model architecture.

5. Model Training Methodology

YOLOv8 Small selection

YOLOv8 Small (YOLOv8s) was selected because it balances real-time inference capability, lower computational requirements, strong detection accuracy, edge-deployment suitability, and better FPS performance than larger variants.

Training configuration	Value
Model	YOLOv8 Small (YOLOv8s)
Dataset size	1,200 images
Epochs	201
Batch size	32
Workers	2
Annotation format	YOLO format
Framework	Ultralytics YOLOv8
Training device	GPU accelerated environment

Why 201 epochs?

Training was conducted for 201 epochs because the dataset was captured in a controlled inspection environment with largely consistent background, bottle positioning, and camera-angle conditions. Monitoring indicated that approximately 200 epochs supported stable convergence and helped the model learn fine-grained bottle quality characteristics without significant overfitting.

Training outcomes

- Stable model convergence.
- Improved detection consistency after data validation and balancing.
- Reliable classification of bottle quality attributes.
- Strong real-time inference behavior under production-like operating conditions.

6. Project Objectives

- Accuracy: reliably detect bottle, proper_fill, under_fill, over_fill, label_ok, label_torn, and label_missing classes.
- Real-time performance: support live inspection with FPS visibility and CUDA/FP16 acceleration where available.
- Scalability: maintain modular frontend, API, inference, stream, analytics, persistence, and reporting boundaries.
- Business readiness: provide dashboards, history, CSV exports, evidence capture, and management-ready summaries.

Objective area	Definition of success	Implemented evidence
Functional coverage	All core workflows usable from the web app	Dashboard, live, upload, analytics, history, settings
Source flexibility	Multiple video input paths accepted	Webcam, RTSP, mobile IP camera, uploaded files
Operational insight	KPIs visible during and after inspection	Stats cards, charts, logs, CSV exports
Maintainability	Clear implementation boundaries	React pages, service helpers, FastAPI routers

7. Solution Architecture

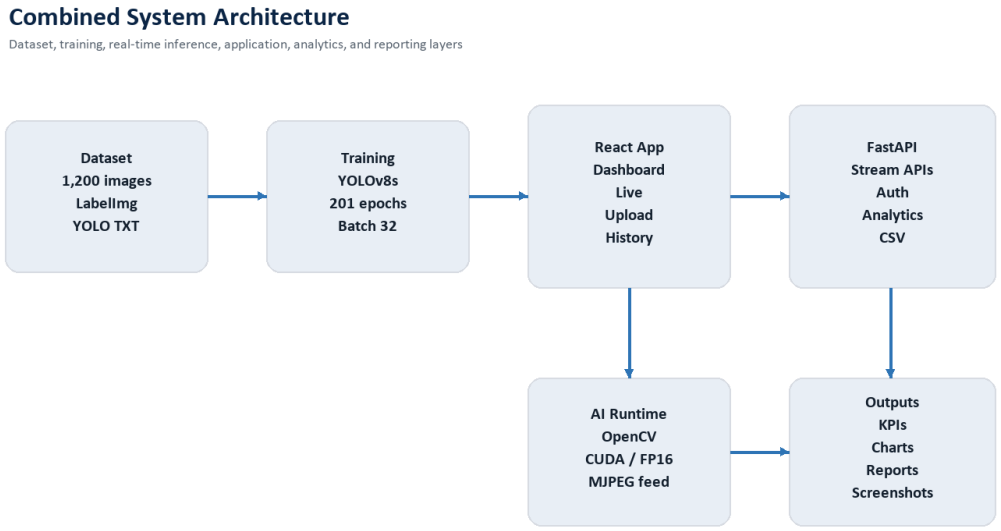

Figure 1. Combined architecture covering dataset, training, app, inference, analytics, and reporting.

The platform separates the React frontend, FastAPI backend, OpenCV/YOLO inference runtime, persistence, and reporting outputs. This structure makes the system easier to debug and prepares it for cloud deployment, edge AI, and multi-camera expansion.

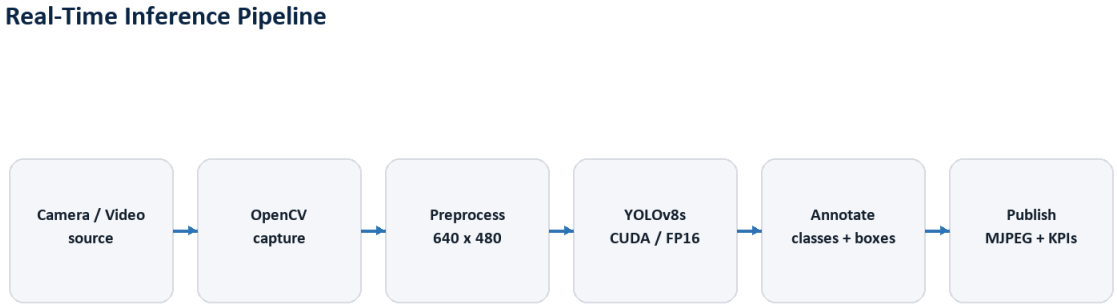

Figure 2. Real-time inference pipeline from source acquisition to browser-visible KPIs.

- React owns user workflows, protected navigation, live controls, dashboard cards, charts, history, and reports.
- FastAPI owns model health, upload inspection, stream control, analytics, history, CSV export, and user APIs.

- OpenCV handles capture, frame sizing, stream access, annotation rendering, and JPEG encoding.
- YOLOv8s performs class detection and supports CUDA/FP16 acceleration in the runtime environment.

8. Technology Stack

Layer	Technology	Role
Frontend	React, Vite, React Router	Operator UI, protected workflows, dashboards
Visualization	Recharts, custom CSS, lucide-react	KPIs, charts, icons, responsive UI
Backend	Python, FastAPI, Uvicorn	REST APIs, stream control, health, history, reports
AI/CV	YOLOv8s, OpenCV, NumPy	Detection, capture, preprocessing, annotation
Acceleration	PyTorch CUDA, FP16	Lower-latency inference
Persistence	SQLite, SQLAlchemy	Inspection records, history, user data
Reporting	CSV, screenshots, DOCX/PDF	Evidence and management review

The stack was selected to support a practical industrial prototype: React for a responsive dashboard, FastAPI for Python-native AI service development, YOLOv8s/OpenCV for mature detection and video processing, and CUDA/FP16 to improve real-time throughput.

9. Features Implemented

- Authentication and protected app routes.
- Dashboard with model status, inspection KPIs, charts, and recent inspections.
- Live Inspection page with webcam, RTSP, and mobile camera source support.
- Uploaded video inspection with annotated stream output.
- Analytics dashboard with pass/fail, defect categories, and trends.
- Inspection history with search, filters, pagination, and report export.
- CSV export for per-inspection and global reporting.
- Admin/support features such as settings, account pages, activity logs, toasts, screenshot capture, and fullscreen viewing.

Feature group	Business value
Live Inspection	Real-time production monitoring
Video Upload	Offline review and validation of recorded footage
Analytics	Management visibility into quality trends
History and CSV Export	Traceability and evidence automation
Screenshot and Fullscreen	Better review and communication during inspections

10. Engineering Challenges

- Maintaining stream stability across webcam, RTSP, and mobile camera sources.
- Optimizing FPS without sacrificing detection quality.
- Handling RTSP buffering, transport, reconnect behavior, and timeouts.
- Normalizing mobile camera IP, port, and stream path inputs.
- Avoiding stale frame backlog in real-time inference.
- Keeping UI state responsive during asynchronous stream startup, stop, reconnect, and screenshot actions.
- Maintaining a balanced dataset and consistent labels before training.

11. Solutions Implemented

Challenge	Solution	Result
Dataset imbalance	Validation and balancing script	Improved training readiness
GPU performance	CUDA/FP16 inference path	Lower latency and smoother live inspection
RTSP instability	Low-latency OpenCV capture settings and retry handling	More resilient network camera support
Frame backlog	Latest-frame publishing approach	Current frames prioritized
Reporting burden	History and CSV exports	Automated evidence trail
Debugging ambiguity	Stage-by-stage tracing	Faster root-cause isolation

The debugging approach emphasized complete end-to-end tracing. For example, screenshot capture was traced from button click to capture function entry, frame detection, canvas creation, drawImage, blob generation, and download triggering. This revealed the difference between downloading an MJPEG endpoint and capturing the current visible frame.

12. Performance Results

The system was functionally validated across YOLO prediction, webcam, RTSP, mobile camera, uploaded video, live inspection, navigation, analytics, history, screenshot capture, and fullscreen workflows.

Metric	Measurement path	Status
Detection accuracy	YOLO detection/class mapping	Validated functionally; formal mAP benchmark recommended
Average FPS	average_inference_fps endpoint	Available for live measurement
Inference latency	inference_time_ms endpoint	Available for bottleneck analysis
Capture latency	capture_time_ms endpoint	Available for stream diagnostics
GPU status	CUDA/device/FP16 flags	CUDA and FP16 supported in runtime
Stability	frames_dropped and last_error	Available for live monitoring

Runtime health checks reported model readiness, CUDA model execution, FP16 support, and NVIDIA GPU availability. The implementation exposes the telemetry needed for formal future benchmarking, including FPS, latency, stream status, frame drops, and GPU state.

13. Business Impact

- Reduces repetitive manual inspection effort.
- Improves consistency of quality-control decisions.
- Provides real-time monitoring for production stakeholders.
- Creates structured inspection history and CSV reports.
- Supports faster defect trend review and root-cause investigation.
- Provides a scalable foundation for multi-camera, cloud, and edge deployment.

14. Key Learnings

- Dataset quality impacts performance more than model size alone.
- Annotation consistency directly affects accuracy.
- Class balancing significantly improves model reliability.
- Real-time AI systems require optimization beyond model training.
- FPS depends on model efficiency, camera handling, frame queues, GPU execution, encoding, and browser streaming.
- Full-stack ownership is required to transform a model into a usable industrial product.

15. Future Roadmap

- Multi-camera production monitoring with per-camera health and KPIs.
- Cloud deployment with managed database, object storage, and observability.
- Advanced analytics including alerts, heatmaps, defect trends, and shift reports.
- Edge AI deployment near production cameras to reduce latency.
- Formal model evaluation suite with mAP, precision, recall, FPS, and latency reporting.
- Role-based access control for operator, manager, and administrator workflows.

16. Conclusion

The AI-Powered Water Bottle Quality Inspection System demonstrates a complete industrial computer vision lifecycle: production-like data collection, LabelImg annotation, dataset balancing, YOLOv8s training, real-time inference optimization, full-stack application development, analytics, history, exports, and management-ready documentation.

The combined project work highlights strong individual technical ownership and problem-solving ability across AI, backend, frontend, performance optimization, production debugging, and business communication. It gives SEEWISE.ai a practical foundation for automated quality inspection and future production deployment.

17. Appendix

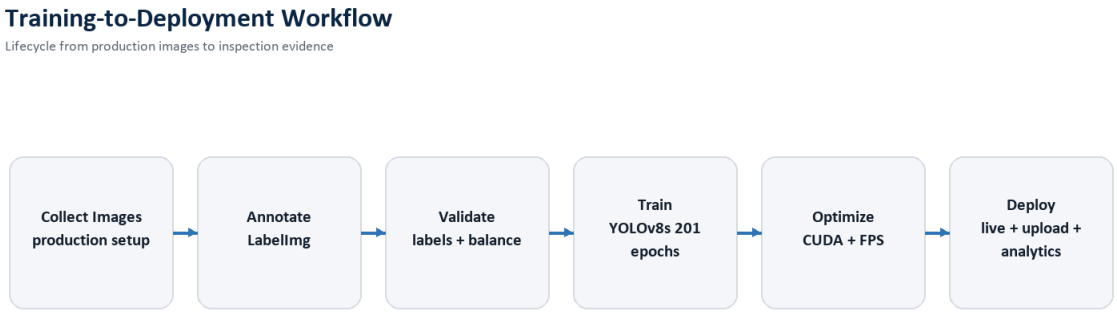

Figure 3. Training-to-deployment workflow for the inspection system.

Detection class	Meaning
bottle	Bottle object detected
proper_fill	Correct fill level
under_fill	Below acceptable fill threshold
over_fill	Above acceptable fill threshold
label_ok	Acceptable label
label_torn	Damaged label
label_missing	Missing label

API area	Representative endpoints
Health	GET /health
Upload inspection	POST /upload-video, POST /start-inference
Live stream	POST /start-stream, GET /stream-status
Streaming output	GET /video-feed, GET /video-feed-stats
Analytics	GET /api/analytics/dashboard
History and reports	GET /api/history/, GET /export-csv
Users	POST /api/users/register, POST /api/users/login